READING SHEET

Data Fetching — SWR & React Query

Module 3 · Frontend: React & Next.js · Day 41 · Week 9

PROGRAM	WORK TYPE	EST. TIME
CodeOps · Full Stack	**Revision + mini-project**	**2–3 hours**

Tip How to use this sheet

Week 8 moved most of your data onto the server. What remains in the browser — polling, search-as-you-type, paging — is the hardest part, because it needs caching, deduplication and protection against an old response arriving last. This sheet covers why a library helps, SWR, TanStack Query, and how to combine a server-rendered first paint with a live client query.

Part 1 · Why a Library

Nothing here reverses Day 38. If a server component can fetch it, let it. This is for the cases where it genuinely cannot — and that list, though short, is the difficult one.

NOW ON THE SERVER	STILL ON THE CLIENT
The menu, a dish, a receipt	Polling an order status
The first paint of any page	Search results as you type
Anything a crawler should see	Pages two onward of a list
Anything needing a secret	Anything refetched on a schedule
Data shared by every visitor	Data only this tab cares about

Everything remaining is triggered by the person, repeated, or both — precisely where a naive effect breaks down.

What the effect version misses

```
                                                the Day 29 pattern — correct, and not enough
useEffect(() => {
  let cancelled = false;
  fetch(url).then(/* ... */);
  return () => { cancelled = true; };
}, [url]);
```

There is nothing wrong with this code — the cleanup flag was the race-condition fix and it works. It simply has no memory: no cache between mounts, no sharing between components, and no idea that two components just asked for the same thing.

FEATURE	WHAT IT SAVES
Caching	The same request is not sent twice
Deduplication	Three components, one network call
Revalidation	Refresh on focus, reconnect or a timer

FEATURE	WHAT IT SAVES
Stale-while-revalidate	Show what you have, update behind it
Race safety	The last response wins, not the last to arrive
Mutation and invalidation	A write refreshes what it affected

> **Tip Not features — corrections**
> Each of these fixes a bug users actually hit: duplicate requests, a stale screen after a save, an old response overwriting a newer one. Writing them yourself is a project, not an afternoon. SWR and TanStack Query differ in size and surface area, not in philosophy — learn one properly and the other takes an hour.

Stale-while-revalidate

MOMENT	WHAT THE PERSON SEES	BEHIND IT
First visit	A loading state	The request goes out
Second visit	The cached data, instantly	A fresh request goes out quietly
Response arrives	The screen updates if it changed	The cache is replaced
Tab refocused	No flicker	Revalidated in the background

Return the stale value immediately, revalidate in the background, replace it if it changed. The person never waits for data you already have — which on an intermittent connection in Addis is the difference between an application that feels broken and one that feels fast.

Part 2 · SWR

The smaller of the two, from the team behind Next.js. One hook, a **key** and a **fetcher** — and the caching, deduplication and revalidation come with it.

app/orders/OrderStatus.jsx

```
"use client";
import useSWR from "swr";
const fetcher = (url) => fetch(url).then((r) => r.json());

const { data, error, isLoading } =
  useSWR(`/api/orders/${id}`, fetcher);

if (isLoading) return <Spinner />;
if (error)     return <p>Could not load the order.</p>;
```

The three branches you wrote by hand on Day 29 are returned by the hook — the cleanup, the cancellation flag and the dependency array are gone. Two components calling useSWR with the same key share one cache entry and one network request, and that deduplication needs no configuration.

Keys do the thinking

```
                                                          the key drives everything
// changes with the search term -> refetch
useSWR(`/api/dishes?q=${query}`, fetcher);

// null key -> do not fetch at all
useSWR(session ? "/api/orders" : null, fetcher);

// an array key for anything structured
useSWR(["/api/dishes", category, page], fetcher);
```

> **Warning Conditional fetching**
> Never wrap a hook in an if. Pass null as the key instead — that is how you say "not yet" without breaking the rules of hooks. Whatever the key depends on is what triggers a refetch: the same idea as an effect dependency list, except it also identifies the cache entry.

Configuration worth knowing

OPTION	WHAT IT DOES
`refreshInterval`	Poll every N milliseconds
`revalidateOnFocus`	Refetch when the tab regains focus
`dedupingInterval`	Ignore duplicate calls within a window
`fallbackData`	Render immediately with server data
`keepPreviousData`	Hold the old page while the next loads

The order-status screen that would have needed a setInterval, a cleanup and a stale-response guard becomes `{ refreshInterval: 5000 }`. Revalidation on focus and on reconnect are on by default — turn them off for data that genuinely never changes, not as a habit.

Writing, and refreshing after

```
                                                  mutate — tell the cache what changed
const { data, mutate } = useSWR("/api/cart", fetcher);

async function addDish(dish) {
  mutate([...data, dish], false);      // show it now
  await fetch("/api/cart", { method: "POST", /* ... */ });
  mutate();                            // refetch and replace
}
```

Update the screen first and reconcile with the server afterwards — the interface feels instant, and the refetch corrects it if the request failed. An optimistic update that never rolls back is a lie on screen, so wrap the write in a try/catch and revert or refetch on error.

Part 3 · TanStack Query

Larger, and worth the size once an application has real write operations. The same stale-while-revalidate idea, with explicit array keys, a mutation system and the best devtools of any React library. It needs a QueryClient in context, so the provider belongs in the client Providers component from Day 38.

useQuery

```
import { useQuery } from "@tanstack/react-query";

const { data, error, isPending } = useQuery({
  queryKey: ["orders", id],
  queryFn:  () => fetch(`/api/orders/${id}`).then(r => r.json()),
  staleTime: 30_000
});
```

Structured keys let you invalidate broadly or narrowly — everything under `["orders"]`, or just this one order. That precision is the main thing you buy over SWR.

staleTime and gcTime

SETTING	MEANING
`staleTime: 0`	Refetch on the next opportunity (the default)
`staleTime: 30_000`	Treat as fresh for thirty seconds
`staleTime: Infinity`	Never refetch unless told to
`gcTime: 300_000`	Keep unused data cached for five minutes

staleTime asks how long before this is worth refetching; **gcTime** asks how long to keep it after nothing is using it. The default staleTime of zero means a refetch on every mount and focus — safe, and often more traffic than you want. This is the same question as a revalidate window on a route: how stale may this be before someone is misled?

Mutations and invalidation

a write, and everything it makes stale

```
const queryClient = useQueryClient();

const { mutate, isPending } = useMutation({
  mutationFn: (dish) => addToCart(dish),
  onSuccess: () => queryClient.invalidateQueries(
    { queryKey: ["cart"] })
});
```

`invalidateQueries` does for the browser cache what `revalidatePath` does for the server cache: it names what a write affected and lets the library refresh it. The submitting flag you managed by hand on Day 33 comes back as `isPending` — wire it straight to the disabled attribute.

Choosing between them

	SWR	TANSTACK QUERY
Size	Very small	Noticeably larger
Writes	mutate, done by hand	A full mutation system
Devtools	Minimal	Inspect every cache entry
Best for	Mostly reads, a few polls	Many writes, complex invalidation

Use SWR in Addis Eats — the client-side work is a poll and a search box, and the smaller library suits it. Use TanStack Query if your capstone has a lot of writing and updating, and install the devtools on the first day if you do.

Part 4 · Server & Client Together

The best shape is not one or the other. The server renders the first view with real data and the library takes over from there — so there is no loading spinner on arrival and no stale screen afterwards.

```
                                                          no spinner on first paint
// server component
const order = await getOrder(params.id);
return <OrderStatus id={params.id} initial={order} />;

// client
useSWR(`/api/orders/${id}`, fetcher, {
  fallbackData: initial, refreshInterval: 5000 });
```

The page arrives complete and indexable, and the polling begins after hydration — the person never sees a loading state for data the server already had. TanStack Query calls this `initialData`, or a full HydrationBoundary when you want to prefetch several queries at once.

Search as you type

```
                                                  debounce, then let the key do the work
const [term, setTerm] = useState("");
const debounced = useDebounce(term, 300);

const { data } = useSWR(
  debounced ? `/api/dishes?q=${debounced}` : null,
  fetcher, { keepPreviousData: true });
```

Three problems, three lines: debouncing stops a request per keystroke, the null key avoids searching for nothing, and keepPreviousData stops the list flashing empty between terms. Typing "tib" then "tibs" fires two requests — the library guarantees the result shown matches the current key, so the race condition you fixed by hand on Day 29 cannot occur.

Loading more

```
                                                  pagination as a key, infinite as a hook
useSWR(`/api/dishes?page=${page}`, fetcher, { keepPreviousData: true });

const { data, size, setSize } = useSWRInfinite(
  (index) => `/api/dishes?page=${index + 1}`, fetcher);
```

Render the first page on the server so it is fast and indexable, and let the client fetch the rest — the standard shape for any long list. For numbered pages, keep the page in the query string as you did on Day 31: a result someone can link to is worth more than a smooth transition.

Assigned Reading

Required

1. **SWR — Getting started** What / why: the hook, keys, the fetcher and the default revalidation behaviour.
2. **TanStack Query — Important defaults** What / why: staleTime, gcTime and refetch behaviour — read before configuring anything.

Optional — go deeper

- **SWR — Mutation** — optimistic updates and refreshing the cache after a write.
- **stale-while-revalidate** — the HTTP caching idea both libraries are named after.

Exercises

Keep the network tab open throughout — most of these are verified by watching requests.

1. Install SWR and put the fetcher in a shared lib/fetcher.js.
2. Convert the order-status screen to useSWR and delete its effect and state.
3. Add refreshInterval and confirm the polling in the network tab.
4. Render the order in a server component and pass it in as fallbackData.
5. Add a debounced search box that uses a null key when the term is empty.
6. Turn on keepPreviousData and watch the list stop flashing between searches.
7. Add paging with the page number in the query string, then write DATA.md.

Mini-Project · Live Data in Addis Eats

> **Tip What you build**
> The three client-side data problems, solved properly: an order-status screen that is server-rendered and then polls, a menu search that debounces and never shows the wrong results, and a paged dish list that does not flash. Every query key and refresh rule written down and justified.

Requirements

- A shared fetcher used by every query, with errors thrown for non-OK responses.
- An order-status route rendered on the server and polled every five seconds.
- fallbackData supplied so no spinner appears on the first paint.
- A debounced search box using a null key for an empty term, with keepPreviousData enabled so the list never flashes empty.
- Paging with the page number in the query string, linkable and shareable.
- DATA.md listing every query, its key, its refresh rule and one line of reasoning.

What to submit

Your GitHub repository with the queries and DATA.md, plus a README note on what the network tab shows while typing.

Check yourself

- Does typing five characters fire one request or five?
- Does the order page show data immediately, with no spinner?
- Do two components asking for the same key produce one network call?
- Does the list keep previous results visible while the next load runs?
- Can you justify every refreshInterval and staleTime in one sentence?